# 35 câu lệnh tạo AI ExamGuard

**Từng bước từ mã nguồn đến ứng dụng Windows EXE**

Mỗi bước dưới đây là một câu lệnh hoàn chỉnh để sao chép vào Codex, không phải lệnh gõ trong Terminal; mở thư mục dự án muốn làm, gửi lần lượt từng bước trong cùng cuộc trò chuyện và chờ hoàn thành trước khi gửi bước tiếp theo.

Nếu tạo mới, dùng thư mục trống; nếu làm tiếp dự án hiện có, giữ nguyên thư mục nguồn để Codex đọc và bổ sung phần còn thiếu; các bước 30 đến 35 dành cho build EXE, chạy thử và bàn giao.

### Bước 01  Chốt yêu cầu dự án

Hãy xây dựng AI ExamGuard trên Windows theo kiến trúc Next.js, React, TypeScript và Tailwind cho giao diện, MediaPipe Face Landmarker xử lý webcam tại máy thí sinh, Python với pywebview và WebView2 chạy desktop, PyInstaller đóng gói EXE, có bốn trang Trang chủ, Thí sinh, Giám thị, Quản trị và đồng bộ nhiều máy qua LAN; trước tiên hãy đọc mã nguồn cùng tài liệu nếu có, lập REQUIREMENTS.md với phạm vi và tiêu chí nghiệm thu, giữ AI ở vai trò cảnh báo để giám thị xác minh, rồi chỉ hoàn thành bước này và báo kết quả thực tế.

### Bước 02  Chuẩn bị môi trường phát triển

Hãy kiểm tra Node.js, npm, Python, kiến trúc Windows và WebView2, đối chiếu phiên bản tương thích với dự án rồi thiết lập môi trường Python .venv, cài thư viện frontend và các thư viện Python cần thiết gồm pywebview cùng PyInstaller, lưu package-lock.json và requirements.txt với phiên bản tái tạo được, tránh thay đổi môi trường toàn máy nếu không cần và báo rõ thành phần nào còn thiếu.

### Bước 03  Tạo cấu trúc và kiểu dữ liệu

Hãy tạo cấu trúc app, components, lib, public/mediapipe, tests và tai-lieu, khai báo kiểu dữ liệu trong lib/exam-types.ts cho phòng, máy, họ tên, số báo danh, trạng thái máy, trạng thái AI, chỉ số khuôn mặt, sự kiện và ngưỡng cảnh báo, dùng mã sự kiện duy nhất cùng khóa phòng kết hợp số máy để tránh trùng giữa các phòng và tạo dữ liệu giả có nhãn demo để dùng trong quá trình phát triển.

## Tạo giao diện và chuẩn bị AI

### Bước 04  Dựng giao diện chung

Hãy tạo giao diện AI ExamGuard bằng tiếng Việt với nền tối, chữ dễ đọc, nút và biểu tượng thống nhất, thanh điều hướng giữa bốn khu vực, màu xanh cho ổn định, vàng cho cần kiểm tra, đỏ cho cảnh báo mức cao, xám cho ngoại tuyến kèm nhãn chữ tương ứng, dùng tài nguyên giao diện cục bộ và bảo đảm bố cục không bị cắt trên cửa sổ desktop thông dụng.

### Bước 05  Tạo trang chủ chọn vai trò

Hãy hoàn thiện trang chủ tại đường dẫn / với tên AI ExamGuard, mô tả ngắn mục đích hỗ trợ giám sát thi, ba nút đi tới /thi-sinh, /giam-thi và /quan-tri, phần giới thiệu luồng thí sinh gửi tín hiệu để giám thị kiểm tra, thông báo video được xử lý tại máy theo cấu hình mặc định và các liên kết điều hướng hoạt động thật.

### Bước 06  Tạo biểu mẫu thí sinh

Hãy xây dựng trang /thi-sinh với các ô họ tên, số báo danh, mã phòng, số máy và địa chỉ máy giám thị, kiểm tra dữ liệu bắt buộc, hiển thị thông báo giám sát và ô xác nhận đã đọc, chỉ cho bắt đầu khi các điều kiện tương ứng đã đạt, đồng thời giữ nguyên danh tính đã chốt trong mỗi phiên giám sát để cảnh báo không bị gán nhầm người.

### Bước 07  Chuẩn bị tài nguyên AI ngoại tuyến

Hãy tạo và chạy script chuẩn bị tài nguyên AI để lưu face_landmarker.task trong public/mediapipe và các tệp WASM cần thiết trong public/mediapipe/wasm, lấy tài nguyên từ nguồn chính thức tương thích với phiên bản @mediapipe/tasks-vision đã cài, ghi lại nguồn cùng phiên bản và kiểm tra tệp tải về hợp lệ, sau đó thay các đường dẫn tải lúc chạy bằng đường dẫn cục bộ để ứng dụng đã đóng gói không cần Internet cho suy luận AI.

## Kết nối webcam và phân tích hành vi

### Bước 08  Kết nối và điều khiển webcam

Hãy bổ sung chức năng kiểm tra thiết bị, xin quyền webcam khi người dùng thao tác, hiển thị video xem trước và các nút bắt đầu hoặc dừng, báo lỗi tiếng Việt khi thiếu camera, camera đang bận hoặc quyền bị từ chối, giải phóng toàn bộ media track khi kết thúc và dùng địa chỉ localhost cho giao diện desktop để đáp ứng điều kiện truy cập webcam.

### Bước 09  Chạy bộ phân tích khuôn mặt

Hãy tích hợp MediaPipe Face Landmarker ở chế độ video để phát hiện tối đa hai khuôn mặt từ webcam, vẽ điểm mốc hoặc khung mặt lên canvas đồng bộ kích thước video, quản lý vòng phân tích để không tạo tác vụ chồng nhau, hiển thị FPS thực tế và các trạng thái đang tải, sẵn sàng, tạm dừng, lỗi, đã dừng, đồng thời chỉ báo AI đang phân tích sau khi có kết quả suy luận hợp lệ.

### Bước 10  Tính chỉ số quan sát

Hãy tạo mô đun chuyển kết quả Face Landmarker thành faceCount, headYaw, headPitch, gazeX, gazeY và fps với quy ước trục rõ ràng, hiển thị các chỉ số phục vụ hiệu chỉnh, xử lý trường hợp thiếu điểm mốc hoặc nhiều khuôn mặt và ghi rõ góc đầu cùng hướng mắt là giá trị ước tính cần thử với webcam thực tế trước khi sử dụng.

### Bước 11  Viết bộ đếm hành vi

Hãy viết lib/behavior-engine.ts để phát hiện không có mặt sau 5 giây, nhiều khuôn mặt sau 2 giây, quay đầu hoặc cúi ngẩng sau 3 giây và lệch hướng mắt sau 5 giây theo ngưỡng có thể chỉnh, chỉ cộng thời gian quan sát hợp lệ, chịu được nhiễu ngắn dưới 400 mili giây, xóa bộ đếm khi khoảng cách khung hình vượt 1500 mili giây và trả về tiến trình từng loại cảnh báo.

## Phát cảnh báo và theo dõi phòng thi

### Bước 12 Tạo và giới hạn cảnh báo

Hãy nối Behavior Engine với bộ tạo sự kiện có mã duy nhất, thời điểm, phòng, máy, họ tên, số báo danh, loại tín hiệu, mức cảnh báo, chi tiết và nguồn phát sinh, áp dụng thời gian nghỉ mặc định 12 giây cho từng loại để giảm lặp, hiển thị tiến trình đạt ngưỡng cùng danh sách cảnh báo tại máy và tránh biến chỉ số ước tính thành kết luận gian lận.

### Bước 13 Theo dõi thao tác trong cửa sổ thi

Hãy bổ sung cảnh báo khi trang mất tập trung, bị ẩn hoặc nhận thao tác bàn phím sao chép, dán và in trong phạm vi ứng dụng, phân loại chúng là sự kiện trình duyệt, chống phát lặp liên tục và giải thích trong giao diện rằng ứng dụng chỉ ghi nhận được các sự kiện mà trình duyệt hoặc WebView cung cấp, không bảo đảm phát hiện mọi thao tác của hệ điều hành.

### Bước 14 Tạo chế độ thử cục bộ

Hãy xây dựng lớp lưu trữ demo bằng localStorage và BroadcastChannel để hai tab cùng trình duyệt và cùng nguồn địa chỉ trao đổi trạng thái máy, cảnh báo và kết quả xác minh, tách rõ dữ liệu demo với dữ liệu phiên thật, có thao tác xóa riêng dữ liệu demo và gắn nhãn cho người dùng biết chế độ này chỉ phục vụ thử trên một máy.

### Bước 15 Xây dựng màn hình giám thị

Hãy hoàn thiện trang /giam-thi với sơ đồ máy theo phòng, tổng số máy đang kết nối, số cảnh báo chưa xác minh, luồng sự kiện mới nhất và bộ lọc phòng, mức cảnh báo, trạng thái xác minh cùng tìm kiếm thí sinh, bảo đảm mỗi ô máy và mỗi sự kiện hiển thị đúng phòng, số máy, họ tên, số báo danh và thời gian cập nhật.

## Hoàn thiện tính năng và bản desktop

### Bước 16 Xác minh và xuất báo cáo

Hãy bổ sung cửa sổ chi tiết sự kiện, thao tác đánh dấu đã xác minh và nút xuất báo cáo CSV có tiếng Việt đọc được trong Excel, gồm thời gian, phòng, máy, số báo danh, họ tên, loại cảnh báo, mức độ, chi tiết và trạng thái xác minh, xử lý dấu phẩy, dấu ngoặc kép, xuống dòng cùng dữ liệu có thể bị hiểu thành công thức và kiểm tra tệp xuất bằng một bộ dữ liệu mẫu.

### Bước 17 Tạo màn hình quản trị

Hãy hoàn thiện trang /quan-tri để quản lý cấu hình phòng, số máy và các ngưỡng thời gian kể cả cooldown, kiểm tra giá trị hợp lệ, có khôi phục mặc định và xuất cấu hình JSON, để lưu ảnh cảnh báo tắt mặc định và chỉ cho bật khi đã có chức năng lưu thật cùng thông báo phù hợp, đồng thời thể hiện rõ cấu hình đang áp dụng cho máy hay phiên nào.

### Bước 18 Xuất giao diện web thành tệp tĩnh

Hãy cấu hình next.config.ts với output export và xử lý ảnh phù hợp cho bản tĩnh, chạy bản build frontend để tạo out, kiểm tra đủ các trang HTML, JavaScript, CSS, mô hình và WASM, thay tài nguyên CDN hoặc phông chữ tải từ Internet bằng tài nguyên cục bộ và sửa các chức năng phụ thuộc máy chủ Next.js để bản desktop phục vụ được toàn bộ giao diện từ out.

### Bước 19 Tạo chương trình Python desktop

Hãy tạo app_desktop.py dùng pywebview với WebView2 và ThreadingHTTPServer để phục vụ thư mục out, mở giao diện qua 127.0.0.1 với cổng thực tế, hỗ trợ các đường dẫn /thi-sinh, /giam-thi, /quan-tri cùng MIME đúng cho WASM, giới hạn đường dẫn tệp trong thư mục tài nguyên, phân giải tài nguyên khi chạy mã nguồn hoặc PyInstaller và dừng máy chủ khi đóng hết ứng dụng.

## Kết nối nhiều máy qua LAN

### Bước 20 Hoàn thiện cửa sổ và quyền thiết bị

Hãy hoàn thiện luồng quyền webcam trong WebView2 theo API của phiên bản pywebview đang dùng, hiển thị hướng xử lý khi thiếu runtime hoặc bị từ chối quyền, thêm nút Mở Giám thị để tạo cửa sổ riêng giữ cửa sổ thí sinh tiếp tục phân tích, giới hạn cầu nối JavaScript và Python vào thao tác cần thiết rồi chạy thử mở đóng hai cửa sổ cùng giải phóng camera.

### Bước 21 Tạo trung tâm cảnh báo LAN

Hãy bổ sung trung tâm LAN trong app_desktop.py với cổng mặc định 8765 và cổng dự phòng khi bận, các API health, runtime, snapshot, nhận trạng thái máy, nhận cảnh báo và cập nhật xác minh, kiểm tra dữ liệu đầu vào, giới hạn kích thước yêu cầu, khóa dữ liệu dùng chung giữa các luồng, chống trùng mã sự kiện và công bố địa chỉ IP cùng cổng thực tế để máy thí sinh kết nối.

### Bước 22 Kết nối thí sinh với máy giám thị

Hãy thêm bộ kết nối LAN trong lib/exam-store.ts để mỗi máy thí sinh vẫn mở giao diện và webcam trên localhost của chính nó nhưng gửi trạng thái cùng cảnh báo tới địa chỉ máy giám thị đã nhập, có nút Kiểm tra kết nối đối chiếu đúng dịch vụ AI ExamGuard, quản lý thời gian chờ và chỉ báo kết nối thành công khi có phản hồi hợp lệ.

### Bước 23 Bảo đảm gửi lại khi mất mạng

Hãy bổ sung hàng đợi cảnh báo lưu bền tại máy thí sinh và cơ chế thử gửi lại có khoảng chờ khi mất kết nối, giữ nguyên mã sự kiện để máy giám thị không tạo bản trùng, tách hàng đợi theo phiên và địa chỉ đích để tránh gửi nhầm, đồng thời chỉ hiển thị Giám thị đã nhận sau phản hồi xác nhận của trung tâm LAN và hiển thị số sự kiện còn chờ.

## Theo dõi trạng thái và kiểm thử

### Bước 24 Phân biệt kết nối mạng và hoạt động AI

Hãy hoàn thiện heartbeat cùng thời điểm nhận tại máy giám thị, cập nhật sơ đồ phòng từ snapshot LAN và hiển thị riêng trạng thái mạng với trạng thái phân tích AI, đánh dấu máy ngoại tuyến khi quá hạn và báo rõ camera hoặc AI bị dừng dù mạng vẫn kết nối, đồng thời hiển thị kết quả xác minh nhất quán và thông báo dữ liệu trung tâm đang lưu trong bộ nhớ cần xuất trước khi đóng.

### Bước 25 Giữ lựa chọn Firebase tùy chọn

Hãy bổ sung bộ đồng bộ Firebase tùy chọn qua biến môi trường mẫu và hai collection examStations, examEvents trong cùng lớp lưu trữ, giữ chế độ LAN hoạt động khi không có cấu hình Firebase, hiển thị rõ chế độ kết nối đang dùng, kiểm thử bộ đồng bộ bằng Firebase Emulator nếu khả dụng và ghi riêng yêu cầu xác thực, phân quyền cùng Security Rules trước khi dùng dịch vụ thật.

### Bước 26 Kiểm thử logic cảnh báo

Hãy viết và chạy kiểm thử tự động cho Behavior Engine cùng bộ tạo sự kiện, bao phủ chưa đủ ngưỡng, vừa đủ ngưỡng, hồi phục, nhiễu ngắn, khoảng gián đoạn khung hình dài, nhiều khuôn mặt, cooldown, reset phiên và bảo toàn danh tính, kiểm tra thời gian bằng đồng hồ giả lập có kiểm soát rồi sửa lỗi cho đến khi các ca kiểm thử có ý nghĩa đều đạt.

### Bước 27 Kiểm thử máy chủ và dữ liệu LAN

Hãy viết và chạy kiểm thử Python cho định tuyến trang tĩnh, MIME của WASM, API health, nhận trạng thái và cảnh báo, từ chối dữ liệu lỗi, chống ghi trùng khi gửi lại, xác minh sự kiện, cổng bị chiếm và giới hạn truy cập tệp, đồng thời kiểm tra hai phòng có cùng số máy không bị ghi đè và lưu kết quả chạy vào báo cáo kiểm thử.

## Chốt mã nguồn và chuẩn bị đóng gói

### Bước 28 Kiểm thử toàn bộ luồng ứng dụng

Hãy chạy kiểm thử tích hợp bằng hai tiến trình hoặc hai cửa sổ với dữ liệu QA riêng để chứng minh cảnh báo từ thí sinh đến đúng giám thị, danh tính đúng, xác minh cập nhật được và CSV xuất đúng, thử mất mạng rồi gửi lại, chuyển cửa sổ và dừng camera, ghi rõ phần dùng camera giả hoặc chỉ số giả lập cùng những ca cần người dùng thử tiếp bằng webcam thật hoặc hai máy LAN thật.

### Bước 29 Sửa lỗi và chốt bản nguồn

Hãy chạy lint, kiểm tra kiểu TypeScript, các bộ kiểm thử JavaScript và Python cùng build frontend, sửa mọi lỗi thuộc phạm vi AI ExamGuard rồi chạy lại các kiểm tra bị ảnh hưởng, kiểm tra riêng ngoại tuyến không Internet nhưng còn LAN và trạng thái AI ngừng phân tích, sau đó ghi phiên bản ứng dụng cùng kết quả thực tế vào CHANGELOG.md và báo các mục chưa thể xác minh.

### Bước 30 Tạo script chạy và build trên Windows

Hãy tạo chay_ung_dung.bat và dong_goi_exe.bat dùng đường dẫn được đặt trong dấu ngoặc kép, tự chuyển về thư mục dự án và ưu tiên Python trong .venv, để script build lần lượt kiểm tra thư viện, build frontend, xác nhận tài nguyên out rồi gọi PyInstaller, dừng ngay khi một công đoạn lỗi và in đường dẫn đầu ra khi thành công mà không yêu cầu người dùng gõ thêm lệnh thủ công.

### Bước 31  Tạo cấu hình đóng gói EXE

Hãy tạo AI_ExamGuard_Portable.spec lấy app_desktop.py làm điểm vào, đóng gói toàn bộ out gồm giao diện và tài nguyên MediaPipe vào một EXE không hiện console với tên mặc định AI_ExamGuard_Portable, chỉ thêm hidden import hoặc dữ liệu phụ khi thư viện thực sự cần, hỗ trợ EXAMGUARD_EXE_NAME để đặt tên phiên bản và kiểm tra đường dẫn tài nguyên ở cả mã nguồn lẫn bản đóng gói.

## Build EXE và bàn giao

### Bước 32  Thực hiện build thành EXE

Hãy thực sự chạy dong_goi_exe.bat trên Windows để tạo dist/AI_ExamGuard_Portable.exe từ mã nguồn vừa kiểm tra, đọc log và sửa nguyên nhân nếu build thất bại, dùng tên phiên bản mới khi EXE cũ đang chạy thay vì ghi đè, rồi kiểm tra tệp đã tồn tại, ghi dung lượng, thời gian build, mã SHA256 và trả về đường dẫn tuyệt đối của đúng EXE vừa tạo.

### Bước 33  Chạy thử chính tệp EXE đã build

Hãy mở đúng EXE vừa tạo và kiểm tra bốn màn hình, quyền webcam, tải mô hình cục bộ, trạng thái AI, cửa sổ giám thị riêng, luồng nhận cảnh báo, xác minh, CSV và đóng ứng dụng, thử lại khi ngắt Internet nhưng giữ LAN nếu môi trường cho phép, đối chiếu phiên bản hiển thị và ghi rõ ca đạt, ca lỗi hoặc ca cần thử trên máy Windows sạch có WebView2.

### Bước 34  Chuẩn bị bộ bàn giao có thể tái tạo

Hãy tạo thư mục bàn giao chứa EXE đã kiểm tra, hướng dẫn chạy, mã nguồn frontend và Python cần thiết, khóa phiên bản thư viện cùng script build, bảo đảm mọi bản sao giao diện được lấy từ cùng lần build mới nhất, loại dữ liệu thí sinh thật, khóa bí mật và thư mục phát triển không cần thiết, rồi kiểm tra bộ mã nguồn bàn giao có đủ tài nguyên để người nhận tự build lại.

### Bước 35  Viết hướng dẫn và xác nhận bàn giao

Hãy viết HUONG_DAN_CHAY_EXE.txt bằng tiếng Việt cho một máy và nhiều máy LAN, nêu cách mở máy giám thị trước rồi nhập địa chỉ thực tế trên máy thí sinh, yêu cầu WebView2 và quyền camera, cách cho phép ứng dụng qua Firewall trên mạng Private, xuất CSV trước khi đóng trung tâm và xử lý lỗi thường gặp, sau đó báo đường dẫn EXE, bộ mã nguồn, kết quả kiểm thử cùng những bước xác minh thực tế còn cần hoàn thành.